Employing Metaprogramming: A Macro-Driven Strategy to Overcome Compiler-Imposed Limitations on Specialization

Davide Cerato
Id. Number: 984481

BSc in Computer Science

Advisor: Prof. Walter Cazzola
Co-Advisor: Mr. Federico Bruzzone





UNIVERSITÀ DEGLI STUDI DI MILANO
Computer Science Department
ADAPT-Lab

Academic Year 2024–2025

COMPUTER SCIENCE DEPARTMENT • ADAPT-LAB

Premise A core challenge in systems programming is achieving zero-cost abstraction through generic programming while permitting type-specific optimizations, known as specialization. This is particularly relevant in Rust, where the trait system ensures performance via monomorphization. However, maximum efficiency occasionally requires to utilize some custom optimized implementations, through *Trait Specialization*.

State of the Art Many languages have explored specialization mechanisms with varying degrees of success. Without native support, developers often rely on complex workarounds that yield cumbersome code or insufficient optimization. In 2015, Rust introduced experimental trait specialization, which soon received widespread interest. However, it remains confined to the experimental Nightly channel due to persistent design and implementation challenges.

Limitations Adopting cutting-edge features involves a tension between innovation and stability, often forcing developers to choose between forgoing performance benefits or committing to volatile compiler versions. In Rust, the Nightly-Stable divide prevents the adoption of specialization in production-ready codebases, effectively blocking its significant potential for optimization and cleaner design in commercial environments.

Problem Statement While specialization is crucial for optimization, its application is restricted by compiler stability concerns. This creates a performance ceiling on stable code, putting a gap between theoretical possibilities and practical availability. Specifically for Rust, the central problem is the absence of a robust, idiomatic and efficient mechanism for implementing trait specialization within the stable ecosystem.

Proposal To address these limitations, this thesis proposes moving specialization logic to an earlier compilation stage. We introduce a metaprogramming-based technique that simulates compiler-implemented specialization by generating the necessary logic at compile time. This allows users to define specialized implementations in stable code with zero runtime overhead. This approach is also applicable to any other language with strong metaprogramming support.

Contributions This work provides two key contributions: a novel metaprogramming-based approach for achieving stable specialization and the implementation of reusable tools. These tools include a Rust procedural macro library serving as an alternative to nightly specialization and an analysis tool designed to identify potential optimization benefits in real-world codebases.

Example Let's consider a trait `MyTrait` with a method `my_method` and we want to have a generic implementation as well as provide a specialized implementation for the type `i32`. In standard Rust this would not be possible due to the lack of stable specialization features.

To apply our tool to this problem, the default implementation does not need any change compared to standard code. The specialized implementation instead, is annotated with the attribute macro `#[when(...)]`, that lets us specify the conditions under which that implementation should be used. In this case, we want to specialize the implementation of `MyTrait` for the type `i32`, so we write `#[when(T = i32)]`. This will create a specialized version of the `trait` and of the `impl`, through a process we call *monomorphic specialization*.

After doing this, we also need to instruct the method calls to use the correct version of the method. This is done by using the `spec!` macro, which takes as first argument the method call and as second argument the type to specialize for. This is required because in the macro expansion phase the Rust compiler doesn't give us enough information to automatically determine the types of the variables involved in the method call.

The complete code is shown in Listing 1, where you can see the macros in action. As you can probably understand, this approach preserves the soundness of the type system, since after the monomorphization process we have two completely separate versions of the trait and its implementation, that the compiler can easily distinguish and type check independently.

```rust
trait MyTrait {
    fn my_method(&self);
}
impl<T> MyTrait for T {
    fn my_method(&self) {
        println!("Generic implementation");
    }
}
#[when(T = i32)]
impl<T> MyTrait for T {
    fn my_method(&self) {
        println!("Specialized for i32");
    }
}
fn main() {
    spec! { 10.my_method(); i32 };   // prints "Specialized for i32"
    spec! { 10.5.my_method(); f64 }; // prints "Generic implementation"
}
```

Listing 1. *Example of the tool usage.*

With this simple example, we have shown how to use the tool to define a specialized implementation of a trait method for a specific type. More complex scenarios are also supported, including multiple specializations, constraints on generic parameters, trait bounds and more.